**Library Management System**

Submitted in partial fulfilment of the requirements for the course UI/UX Design

II/IV B.Tech CSE (Data Science) – VI Semester

Submitted by
Mogili Pavan Eswar (Y23CD124)

March – 2026

**R. V. R. & J. C. College of Engineering**
(Autonomous)
NAAC A+ Grade) (Approved by A.I.C.T.E.)
(Affiliated to Acharya Nagarjuna University)
Chandramoulipuram : : Chowdavaram
Guntur – 522 019

# ACKNOWLEDGEMENT

I would like to express my sincere gratitude to everyone who supported me during the completion of my UI/UX Design assignment titled "**Pado Aur Seekho – Library Management System**."

First and foremost, I would like to thank Mr. K. Sai Prasanth, faculty of the UI/UX Design course, for giving me the opportunity to work on this assignment and for his valuable guidance and encouragement throughout the development of this project. His suggestions and support helped me understand the concepts of UI/UX design and implement them effectively in this application.

I would also like to thank the Department of Computer Science and Engineering (Data Science), **R.V.R. & J.C. College of Engineering**, for providing the necessary academic environment and resources that helped me complete this project successfully.

Finally, I would like to express my heartfelt thanks to my family and friends for their constant support, motivation, and encouragement throughout the process of completing this assignment.

**MOGILI PAVAN ESWAR**
**(REG: Y23CD124)**

# ABSTRACT

The Pado Aur Seekho – Library Management System is a web-based application designed to simplify and digitalize the process of managing library resources. The system provides an interactive platform where students can browse available books, request books for borrowing, and monitor the status of their requests. Librarians are provided with administrative controls to review requests, approve or reject them, and update book returns.

The application is developed using HTML, CSS, and JavaScript, focusing on a clean and user-friendly interface based on UI/UX design principles. Browser-based LocalStorage is used to store and manage data such as book records, request details, and user sessions. The system also supports features such as real-time book availability tracking, return deadlines, and overdue fine calculation.

This project demonstrates how effective UI/UX design and web technologies can be used to create a simple, efficient, and accessible library management system that improves both user experience and administrative efficiency.

# CONTENTS


| | Page |
|---|---|
| CHAPTER - 1 : Phases of UI/UX Design | 01 |
| CHAPTER - 2 : Application Explanation | 03 |
| CHAPTER - 3 : Technologies Used | 05 |
| CHAPTER - 4 : Explanation of APIs and Methods Used | 07 |
| Conclusion | 10 |

# CHAPTER - 1 :
# Phases of UI/UX Design

## User Requirement Analysis:

The first phase of the UI/UX design process involved identifying the requirements of the users who would interact with the system. The primary users of the application are students and librarians. Students require an easy way to search for books, request books, and check their borrowing status. Librarians need tools to manage book inventory, review requests, approve or reject them, and update book returns. By understanding these user requirements, the system was designed to provide clear functionality for both types of users.

## Information Architecture:

Information architecture refers to the way information and features are organized within the application. The system was structured into multiple sections such as Login Page, Home Page, Books Page, Request Book Page, Status Page, and Librarian Management Panels. Each section was designed to perform a specific task so that users can navigate the system easily. The navigation bar placed at the top of each page helps users move between different sections quickly and efficiently.

## Information Architecture:

Information architecture refers to the way information and features are organized within the application. The system was structured into multiple sections such as Login Page, Home Page, Books Page, Request Book Page, Status Page, and Librarian Management Panels. Each section was designed to perform a specific task so that users can navigate the system easily. The navigation bar placed at the top of each page helps users move between different sections quickly and efficiently.

## Low-Fidelity Wireframing:

Before implementing the final design, low-fidelity wireframes were created to visualize the layout of the application. These wireframes focused on the placement of key elements such as navigation menus, input fields, buttons, and tables. The goal of wireframing was to ensure that the interface remains simple and organized. By planning the structure in advance, it became easier to design the final user interface without unnecessary complexity.

## Visual Interface Design:

The visual design of the application was developed using HTML and CSS. The interface was designed with a consistent theme and layout across all pages. A navigation bar was included at the top of each page to maintain uniformity. Tables were used to present book details and request information in a structured format. The use of colors, spacing, and alignment helps users easily read and understand the information displayed on the screen.

## User Experience Optimization:

User experience optimization was an important part of the design process. The system was designed to minimize the number of steps required to perform an action. For example, students can request multiple books directly from the books page, and librarians can approve or reject requests with a single click. Status indicators such as Pending, Approved, Rejected, Returned, and Overdue were added to clearly communicate the current state of a request.

## Testing and Evaluation:

After completing the design and development phases, the application was tested to ensure that all features work correctly. Test scenarios included student login, book browsing, sending requests, librarian approval, and book return updates. Testing helped identify any usability issues and ensured that the system provides a smooth and efficient experience for both students and librarians.

# CHAPTER - 2 :
# Application Explanation

The **Pado Aur Seekho Library Management System** is a web-based application developed to simplify the process of managing library books and student requests. The system allows students to browse available books, request books for borrowing, and track the status of their requests. Librarians can review requests, approve or reject them based on book availability, and update the system when books are returned.
The application provides a digital platform that replaces manual record keeping with a more organized and efficient system. By automating tasks such as request approval, book tracking, and return updates, the system improves overall library management.

## Login Module:

The login module is the entry point of the application. Users can log in either as a student or a librarian. The login system verifies the credentials entered by the user and assigns the correct role. Based on the role, the system displays the appropriate features and navigation options. This role-based access control ensures that students and librarians only access the functions that are relevant to them.

## Home Page:

After successful login, users are redirected to the home page of the application. The home page provides a simple overview of the system and allows users to navigate to different sections using the navigation bar. The navigation menu includes links such as Books, Request Book, Status, Requests Panel, and Return Books, depending on the role of the user.

## Books Module:

The Books page displays the list of books available in the library. Each book entry includes details such as Book ID, Book Title, Author Name, Branch or Department, and the number of available copies. Students can browse through the list of books and select the books they wish to borrow.
If the number of copies for a particular book reaches zero, the system prevents students from requesting that book. This feature ensures that requests are only made for books that are currently available in the library.

## Book Request Module:

The book request module allows students to send requests for borrowing books. Students can select one or more books and submit a request along with the issue date. The system automatically calculates the return deadline based on the issue date.
Once the request is submitted, the request is stored in the system and marked with a Pending status. The request then appears in the librarian's request management panel for further action.

## Librarian Request Management:

The librarian has access to a special request management panel where all student requests are displayed. The librarian can review each request and decide whether to approve or reject it. Before approving a request, the system checks the number of available copies of the requested book.

If the request is approved, the system automatically decreases the number of available copies for that book. If the request is rejected, the system updates the request status accordingly. This process ensures proper control over book inventory.

## Book Return Management:

When students return borrowed books to the library, the librarian updates the system using the Return Books page. This page displays all approved book requests that have not yet been returned.

The librarian can mark a book as returned by clicking the Mark Returned button. Once a book is marked as returned, the system increases the number of available copies in the book inventory. This ensures that the library records remain accurate and up to date.

## Status Tracking System:

Students can monitor their borrowing activity through the Status page. This page displays the history of their book requests along with important information such as issue date, return deadline, request status, and overdue fines.

Different status indicators such as Pending, Approved, Rejected, Returned, and Overdue help students understand the current state of their requests. If a book is not returned before the deadline, the system calculates a fine based on the number of overdue days.

## Overall System Workflow:

The overall workflow of the application follows a simple and structured process. Students browse books and send requests for borrowing. Librarians review these requests and approve them if the books are available. When books are returned, librarians update the system, which restores the available copies.

This workflow ensures smooth interaction between students and librarians while maintaining proper control over library resources.

# CHAPTER - 3 :
# Technologies Used

The **Pado Aur Seekho – Library Management System** was developed using a combination of modern web technologies that enable the application to run directly in a web browser. These technologies were selected because they provide a simple, efficient, and flexible way to build interactive web applications. The main technologies used in this project include **HTML, CSS, JavaScript, LocalStorage,** and **Netlify** for deployment.

## HTML (HyperText Markup Language):

HTML is the standard markup language used to create the structure of web pages. It provides the basic framework for displaying content on the internet. In this project, HTML was used to create all the pages of the application such as Login Page, Home Page, Books Page, Request Book Page, Status Page, Librarian Request Panel, and Return Books Page.
HTML elements such as forms, tables, buttons, navigation bars, headings, and input fields were used to organize the content and enable user interaction. Forms were used for login and book request submissions, while tables were used to display book information and request details in a structured format. HTML plays a crucial role in defining how the content of the application is displayed to the user.

## CSS (Cascading Style Sheets)

CSS is used to control the visual appearance and layout of web pages. It allows developers to design attractive and consistent user interfaces. In this project, CSS was used to style the application by defining colors, fonts, spacing, backgrounds, and alignment.
The navigation bar was styled using CSS to provide easy navigation across different pages. Background images and containers were used to create a visually appealing layout. Tables were styled to clearly present book information and request status. Buttons were designed with hover effects and colors to make them interactive and user-friendly. By using CSS, the application maintains a clean and professional appearance across all pages.

## JavaScript:

JavaScript is a powerful programming language used to add interactivity and dynamic behavior to web applications. In this project, JavaScript was used to implement most of the system's functionality. It handles user interactions such as login validation, book requests, request approvals, return updates, and status tracking.
JavaScript is also responsible for updating the page content dynamically without reloading the entire page. For example, when a librarian approves a request, JavaScript updates the request status and reduces the number of available book copies automatically. Similarly, when a book is returned, JavaScript updates the inventory and marks the request as returned.
JavaScript also enables role-based access control, ensuring that students and librarians see only the features that are relevant to their roles.

## LocalStorage:

LocalStorage is a web storage technology that allows data to be stored locally in a user's browser. It is part of the Web Storage API and provides a simple way to store key-value pairs that persist even after the browser is closed.
In this project, LocalStorage acts as a lightweight database for storing information such as book inventory, book requests, user login sessions, and request history. When users interact with the system, the data is saved in LocalStorage so that it can be retrieved later when the application loads again.
Using LocalStorage eliminates the need for a backend server or external database, making the application easier to deploy and run entirely on the client side.

## Netlify Deployment:

Netlify is a cloud-based platform used to host and deploy websites. It allows developers to publish static websites quickly and easily. In this project, Netlify was used to deploy the Library Management System so that it can be accessed online through a public web link.
By uploading the project files to Netlify, the application becomes accessible from any device with an internet connection. Netlify also provides features such as automatic deployment, secure hosting, and fast website performance.

The combination of **HTML, CSS, JavaScript, LocalStorage,** and **Netlify** makes it possible to build a fully functional web application that manages library operations efficiently. These technologies work together to create an interactive interface, manage application logic, store data, and make the system accessible online.

# CHAPTER - 4 :
# Explanation of APIs and Methods Used

The **Pado Aur Seekho – Library Management System** uses various browser APIs and JavaScript methods to manage data, control user interactions, and dynamically update the interface. These APIs and methods allow the application to function efficiently without requiring a backend server or database. The main API used in this project is the LocalStorage API, along with several built-in JavaScript methods for data manipulation and DOM interaction.

## LocalStorage API:

The LocalStorage API is part of the Web Storage API that allows web applications to store data in the browser. It stores data in the form of key-value pairs and keeps the data even after the browser is closed or refreshed. This makes it useful for storing application data such as book inventory, user sessions, and request records.
In this project, LocalStorage is used as a lightweight database. All the information related to books, requests, and user login sessions is stored inside the browser using LocalStorage. When the application loads, the stored data is retrieved and displayed to the user.
Using LocalStorage simplifies the system because it removes the need for an external database or server-side programming.

## setItem() Method:

The setItem() method is used to store data in LocalStorage. It saves information using a key-value pair format.
For example, when a student sends a book request, the request details such as student ID, book ID, issue date, and return date are stored using the setItem() method. Similarly, when a librarian approves a request or marks a book as returned, the updated data is stored again using this method.
This method ensures that the application data remains saved even after the user refreshes the page.

## getItem() Method:

The getItem() method is used to retrieve data that has already been stored in LocalStorage. When the application loads, the system retrieves information such as book lists, requests, and user session data using this method.
For example, when a student opens the Status page, the system uses getItem() to retrieve all stored book requests and filters them to display only the requests belonging to the logged-in student.
This method helps the application display the most recent and updated data stored in the browser.

## removeItem() Method:

The removeItem() method is used to delete stored data from LocalStorage. In this project, it is mainly used during the logout process.
When a user logs out of the application, the stored user session information is removed using removeItem(). This ensures that the next user must log in again before accessing the system.
Removing session data improves the security and proper functioning of the application.

## JSON.stringify() Method:

LocalStorage can only store data in string format. However, application data such as book lists and request records are usually stored as JavaScript objects or arrays. To store these objects in LocalStorage, they must first be converted into strings. The JSON.stringify() method is used to convert JavaScript objects into a JSON string format. This string is then stored in LocalStorage using the setItem() method. For example, when the list of book requests is updated, JSON.stringify() converts the request array into a string so that it can be saved properly in LocalStorage.

## JSON.parse() Method:

The JSON.parse() method performs the reverse operation of JSON.stringify(). It converts a JSON string back into a JavaScript object or array. When the application retrieves stored data using getItem(), the data is returned as a string. JSON.parse() is then used to convert this string back into an object so that the program can manipulate and display the data correctly. This method is essential for reading and processing stored information in the system.

## DOM Manipulation Methods:

The application also uses several DOM (Document Object Model) manipulation methods to interact with HTML elements and update the user interface dynamically.Some commonly used DOM methods in this project include:

**getElementById()**

This method is used to access specific HTML elements by their unique ID. It allows the program to modify the content or style of elements on the page.

innerHTML

This property is used to update the content inside HTML elements. It is commonly used to dynamically generate table rows, display book information, and update request status.

**querySelector() and querySelectorAll()**

These methods are used to select HTML elements based on CSS selectors. They are useful for selecting multiple elements such as checkboxes when students choose books to request.

## Event Handling Methods:

Event handling is used to respond to user actions such as clicking buttons or submitting forms. In this project, event-driven programming is used to perform tasks like sending book requests, approving requests, and marking books as returned.
For example, when a student submits the request form, a JavaScript function is triggered to process the request and store the information in LocalStorage.
Similarly, when a librarian clicks the Approve or Reject button, the corresponding function updates the request status in the system.

## Summary:

The use of the LocalStorage API and JavaScript methods enables the Library Management System to manage data efficiently within the browser environment. Methods such as setItem(), getItem(), removeItem(), JSON.stringify(), JSON.parse(), and DOM manipulation functions work together to store data, retrieve information, and update the user interface dynamically.
These technologies ensure that the application remains lightweight, responsive, and easy to deploy without requiring a complex backend infrastructure.

# CONCLUSION

The Pado Aur Seekho – Library Management System successfully demonstrates how web technologies and UI/UX design principles can be used to develop a simple and efficient library management application. The system provides an easy-to-use interface that allows students to browse available books, request books for borrowing, and track the status of their requests.

The application also provides librarians with tools to manage book requests, approve or reject requests, and update the system when books are returned. Features such as book availability tracking, return deadlines, and overdue fine calculation help improve the efficiency of library operations.

The project was developed using HTML, CSS, and JavaScript, with LocalStorage used for storing application data. The system was deployed using Netlify, making it accessible online through a web browser.

Overall, this project demonstrates how effective UI/UX design and modern web technologies can be used to create a user-friendly digital system that simplifies traditional library management processes. The application improves accessibility, reduces manual record keeping, and provides a better experience for both students and librarians.